Logistic Regression

ICT 3.3 Machine learning and Data Mining

Emanuela Boros, May 2026

Why Logistic Regression?

In linear regression, we predicted numbers i.e. *house price* or *temperature*

→ output = continuous value

What if we want to answer:

- Will it rain today? ☔
- Is this email spam? 📧
- Will this customer buy? 🛒

Logistic Regression

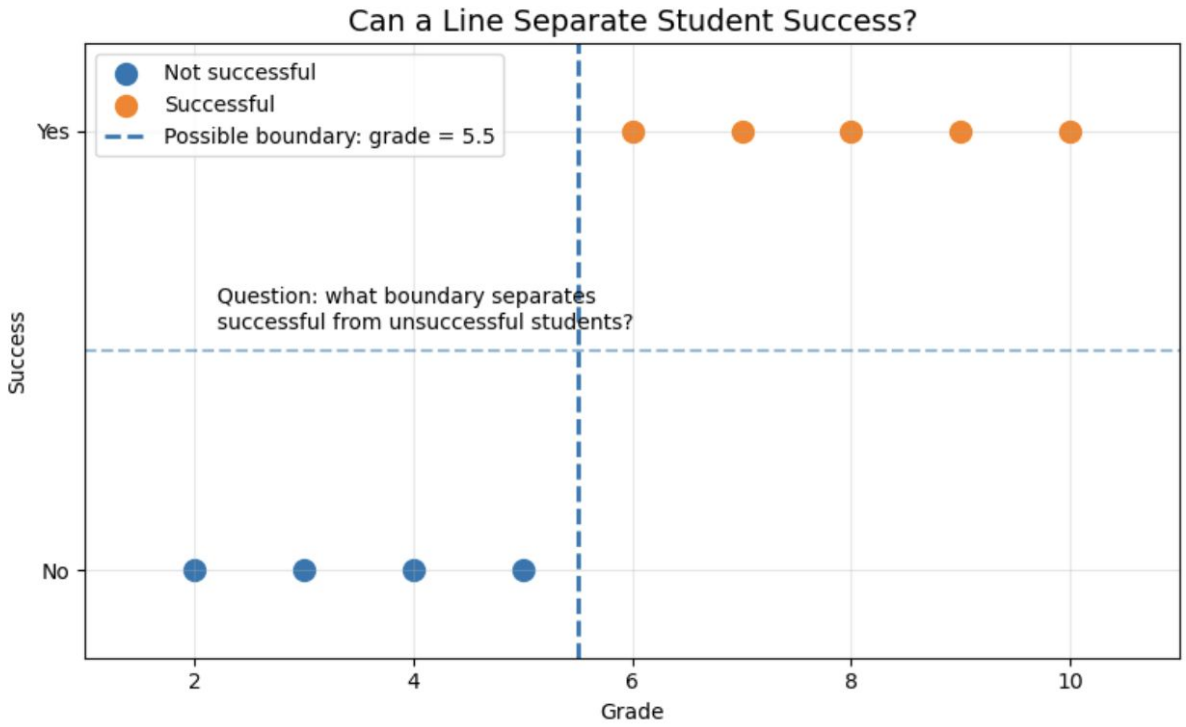
- Used for classification problems
- Predicts a probability (0 → 1)
- Most common case → Binary classification (2 classes)

📧 Email → Spam / Not Spam

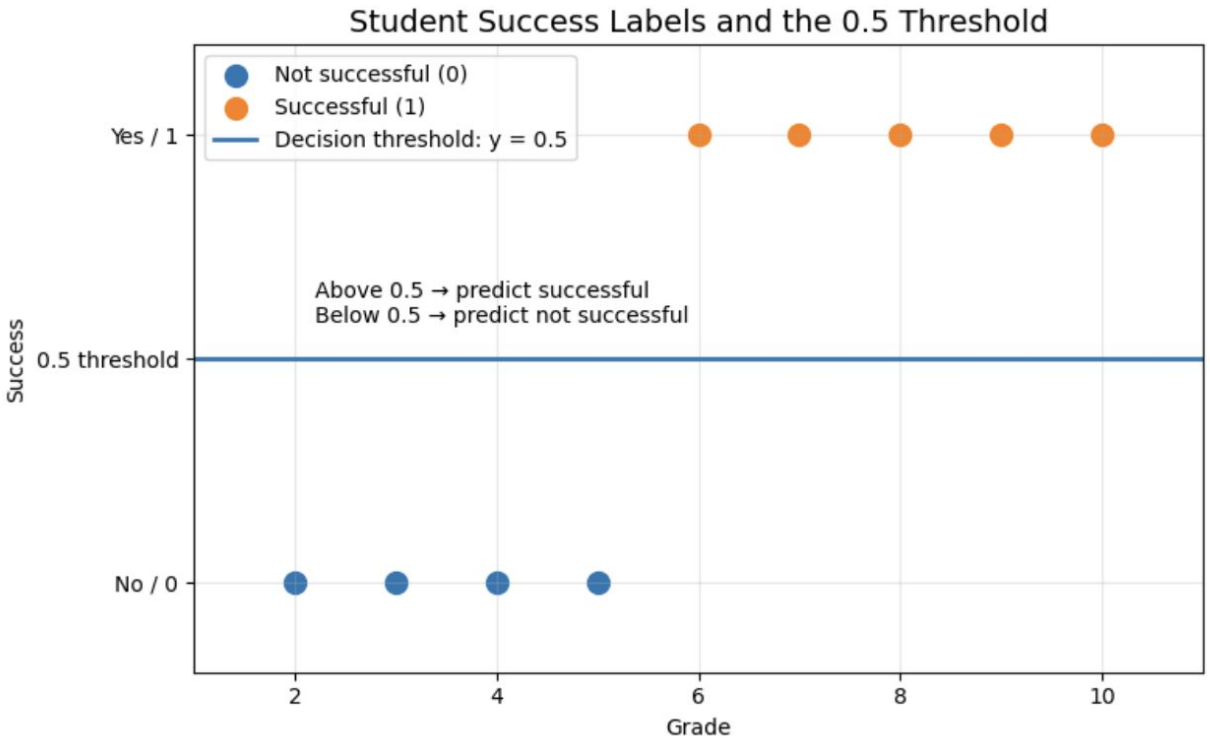
Model output = 0.72

- 72% → spam
- 28% → not spam

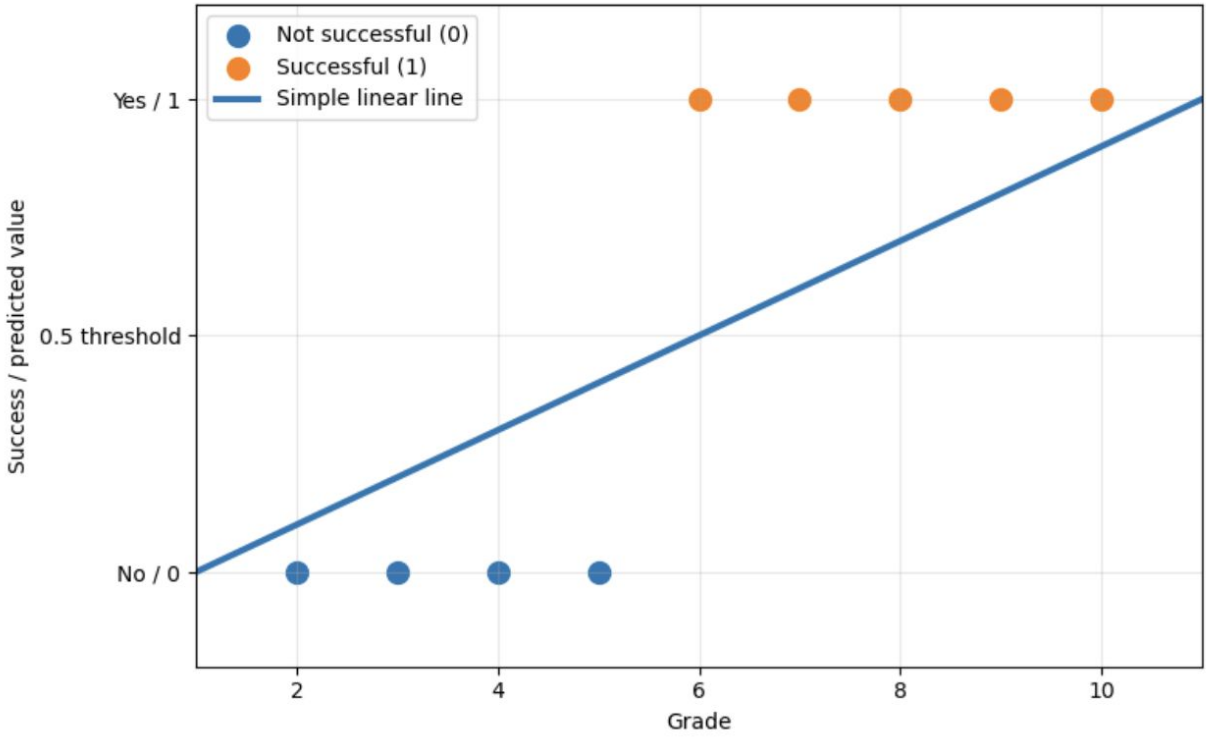
Logistic Regression



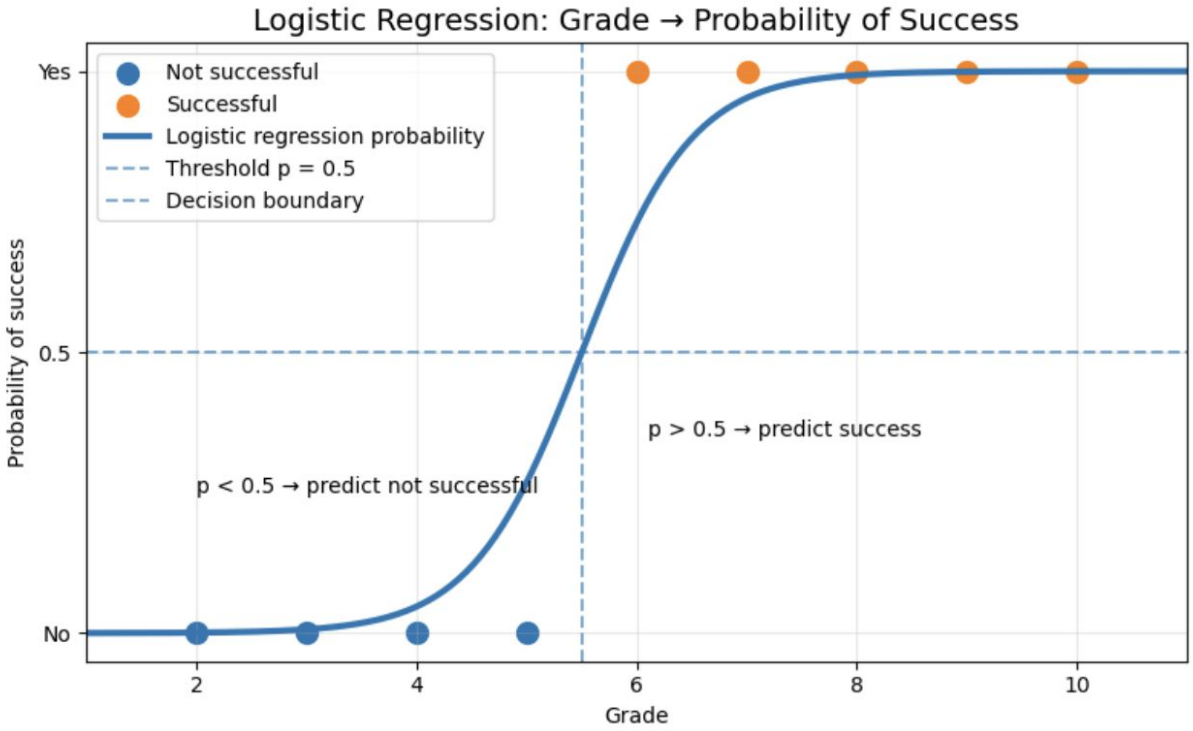
Logistic Regression



Logistic Regression

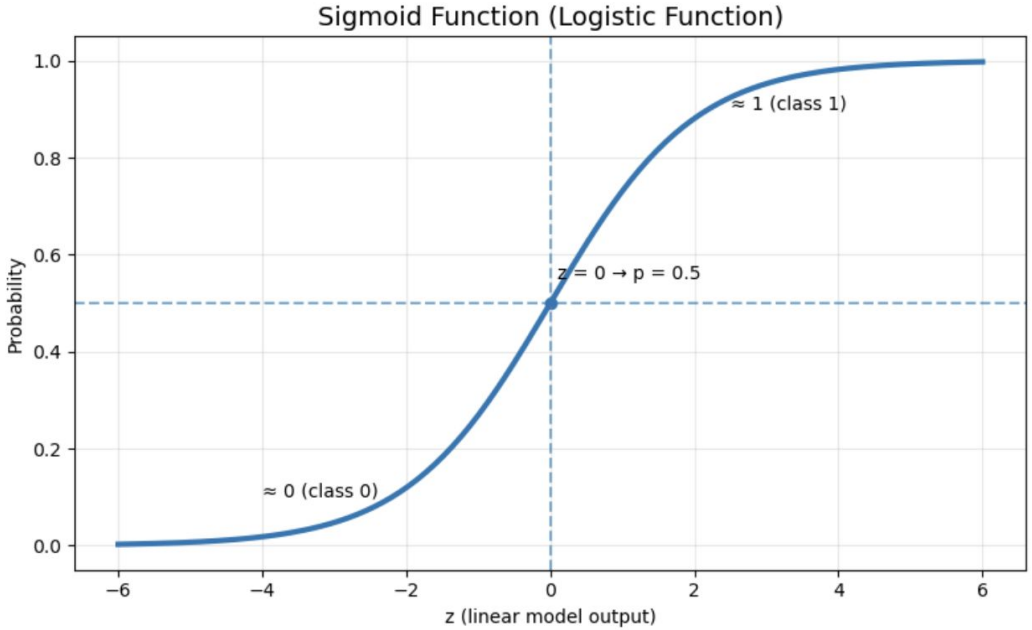


Logistic Regression



Logistic Regression

- doesn’t directly say ‘successful’ or ‘not successful’…
- It gives a probability



Logistic Regression

Linear function:

$$z = wx + b$$

Linear regression

Logistic regression:

$$p(y = 1 \mid x) = \sigma(z)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Probability

Logits

Sigmoid

Logistic Regression

Linear regression:

$$z = wx + b$$

Grade *Weight* Hours *Bias*

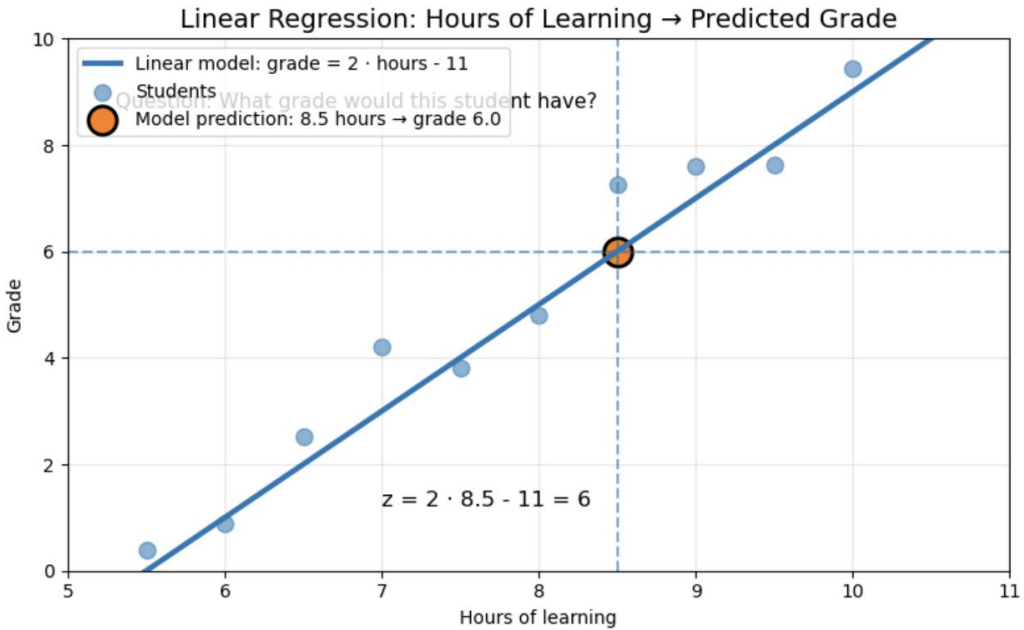
Calculated during training

What grade would this student have?

- w = 2, b = -11
- x = hours of learning = 8.5

$z = 2 \cdot 8.5 - 11 \rightarrow 6$

Logistic Regression



Logistic Regression

Linear regression: $z = wx + b$ $\rightarrow$ grade 6

What is the probability of success of this student?

- w = 2, b = -11
- x = grade = 6

Logistic Regression

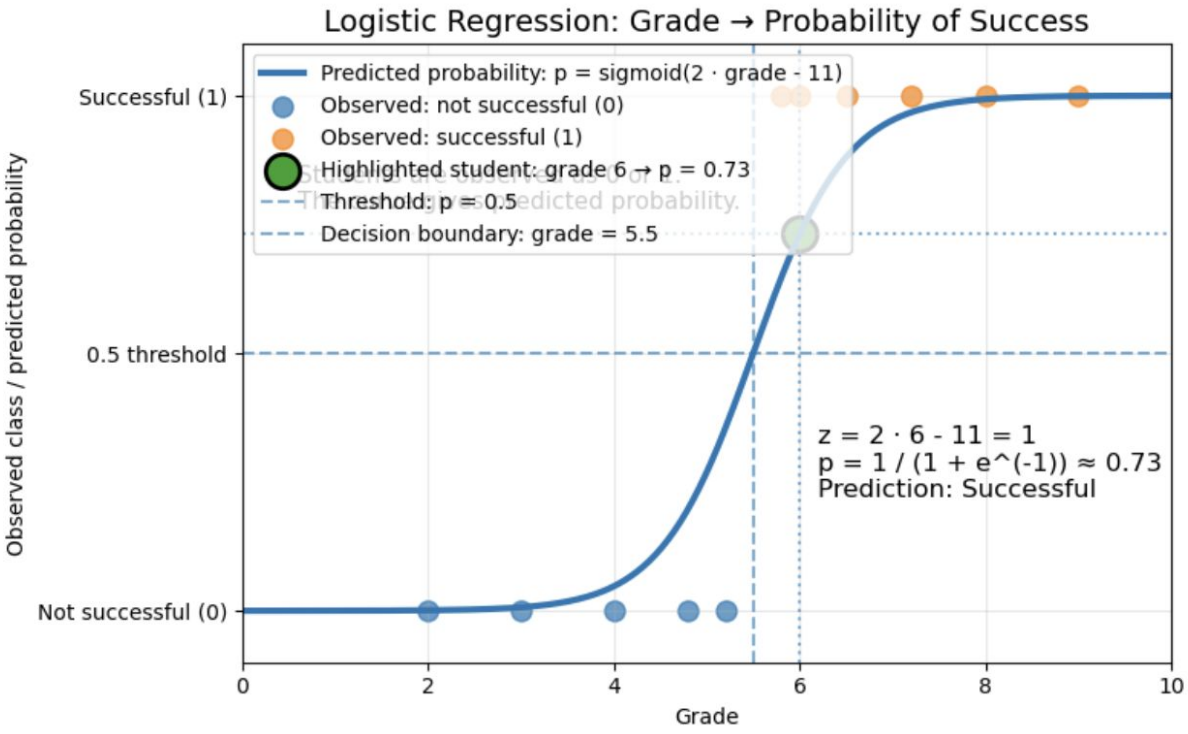
Linear regression: $z = wx + b$ $\rightarrow$ grade 6

What is the probability of success of this student?

- w = 2, b = -11
- x = grade = 6

z = 2 $\cdot$ grade - 11 $\rightarrow$ 1 $\rightarrow p = \dfrac{1}{1 + e^{-1}} \approx 0.73$

Logistic Regression



Measuring Error: Cross-Entropy

In linear regression → we used MSE

→ But now:

- output = probability (0 → 1)
- target = class (0 or 1)

How do we measure error for probabilities?

We need a loss that compares probabilities to true labels → Cross-Entropy (log loss)

Measuring Error: Log Loss

$$\mathcal{L} = -\sum \left[y \log p + (1-y) \log(1-p) \right]$$

We train by minimizing the cross-entropy loss that says if prediction is wrong → loss is large

A model adjusts w to reduce prediction error

If x ≈ 5:

- small weight:
  z = 0.5 · 5 = 2.5 → p ≈ 0.92
- large weight:
  z = 3 · 5 = 15→ p ≈ 1.0

→ larger w makes the sigmoid steeper → more sensitive predictions → If the data is separable, the loss keeps decreasing as $w \to \infty$

Why Not MSE?

MSE does not penalize confident mistakes enough

true y = 1

Case 1 — Good prediction

p = 0.9

MSE: $(1 - 0.9)^2 = 0.01$

→ Small error ✅

Log loss = 0.105

Case 2 — Very bad prediction

p = 0.1

MSE: $(1 - 0.1)^2 = 0.81$

→ Bigger error… but not dramatic ❗

$-\log(0.9) \approx 0.10$

$-\log(0.1) \approx 2.30$

Log loss: 2.30 (much larger!)

Why Not MSE?

MSE does not penalize confident mistakes enough

true y = 1

Case 1 — Good prediction

p = 0.9

MSE: $(1 - 0.9)^2 = 0.01$

→ Small error ✅

Log loss = 0.105

Case 2 — Very bad prediction

p = 0.1

MSE: $(1 - 0.1)^2 = 0.81$

→ Bigger error… but not dramatic ❗

Log loss: 2.30 (much larger!)

In classification, being confident and wrong is dangerous.

Logistic Regression

What about multiple classes?

$y \rightarrow$ {fail, pass, excellent} instead of $y \rightarrow$ {fail, pass} or {success, not success}

Logistic Regression

multinomial logistic regression → called **softmax** regression
(multiclass logistic regression)

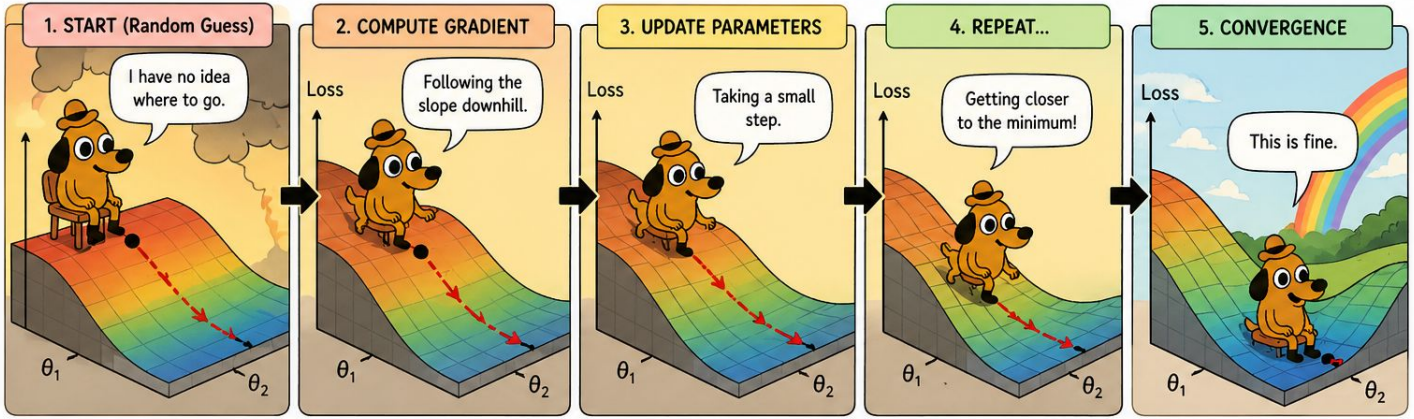
Logistic Regression

0/1 → sigmoid

3+ → softmax → p({fail}) + p({pass}) + p({excellent}) = 1

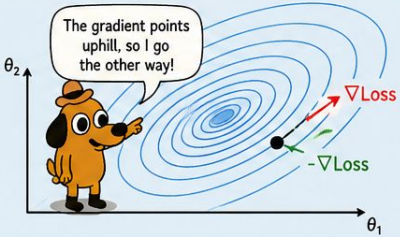
GRADIENT DESCENT: FINDING THE LOWEST LOSS

Goal: Find the parameters (θ) that minimize the loss.



HOW IT WORKS

1. Start with random parameters (θ).
2. Compute the gradient (slope) of the loss.
3. Update parameters in the opposite direction of the gradient.
4. Repeat until the loss is (almost) minimized.



RESULT

Gradient Descent finds the set of parameters that gives the lowest possible loss (local minimum).

Learning rate (α) controls the step size.

Too big? You might overshoot.

Too small? It takes forever.

● = current position

→ = path

Training

model → probability → loss → decision →

Training

model → probability → loss → decision → metrics

Accuracy: How Often Are We Right?

Example

- 100 emails
- 90 correct predictions

→ Accuracy = 90%

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}}$$

percentage of correct predictions

Accuracy tells us how often the model is correct.

Confusion Matrix - Is Accuracy Enough?

- 100 emails
- 95 are NOT spam
- 5 are spam

Predictions: All 100 emails is NOT spam

→ Accuracy = 95% but..

Confusion Matrix - Is Accuracy Enough?

- 100 emails
- 95 are NOT spam
- 5 are spam

Predictions: All 100 emails is NOT spam

→ Accuracy = 95% but.. It never detects spam ❌

Confusion Matrix

FN: This is dangerous → we miss spam.

FP → This is annoying → we block good emails.

		Predictions	
		Spam	~~Spam~~
True	Spam	True positives (TP)	False negatives (FN)
	~~Spam~~	False positives (FP)	True negatives (TN)

The confusion matrix shows what the model gets right and wrong.

Confusion Matrix: Why Does This Matter?

Predict: Does the patient have cancer?

❌ False Negative (FN)

Patient has cancer, but model says “healthy”

❌ False Positive (FP)

Patient is healthy, but model says “cancer”

Some mistakes matter more than others

In this case, we care much more about **avoiding false negatives** → **accuracy** alone is not enough

Recall

Out of all real cases… how many did we catch?

$$\text{Recall} = \frac{TP}{TP + FN}$$

Recall measures how many actual positives we correctly detect

- TP → correctly detected cases
- FN → missed cases

Recall focuses on what we missed

- TP → detected cancer
- FN → missed cancer ❌

80%

Example

- 10 patients have cancer
- model detects 8

$$\text{Recall} = \frac{8}{10} = 0.8$$

Precision

When the model says ‘positive’… how often is it right?”

$$\text{Precision} = \frac{TP}{TP + FP}$$

High precision = few false positives

Precision measures how many predicted positives are actually correct

- TP → correct positive predictions
- FP → false alarms ❌

Example

- Model predicts 10 patients have cancer
- Only 6 actually have it

$$\text{Precision} = \frac{6}{10} = 0.6$$

Precision vs Recall Trade-off

Recall = how many real cases we catch
Precision = how trustworthy our predictions are

Improving recall often reduces precision… and vice versa

- Lower threshold → more positives
  → ✅ higher recall
  → ❌ lower precision
- Higher threshold → fewer positives
  → ❌ lower recall
  → ✅ higher precision

🎯 Goal: detect cancer

- Want high recall → don't miss patients
- Accept lower precision → more false alarms

🎯 Goal: detect fraud

- Want high precision → don't falsely accuse customers
- Accept lower recall → may miss some fraud

F1 Score – Balancing Precision and Recall

F1-score combines precision and recall into a single number

- High only if both precision AND recall are high
- Low if one of them is low

F1 punishes imbalance

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

If one is bad… F1 is bad

Mini Assignment

TODOs:

- Exercise notebook

Clone the repo!